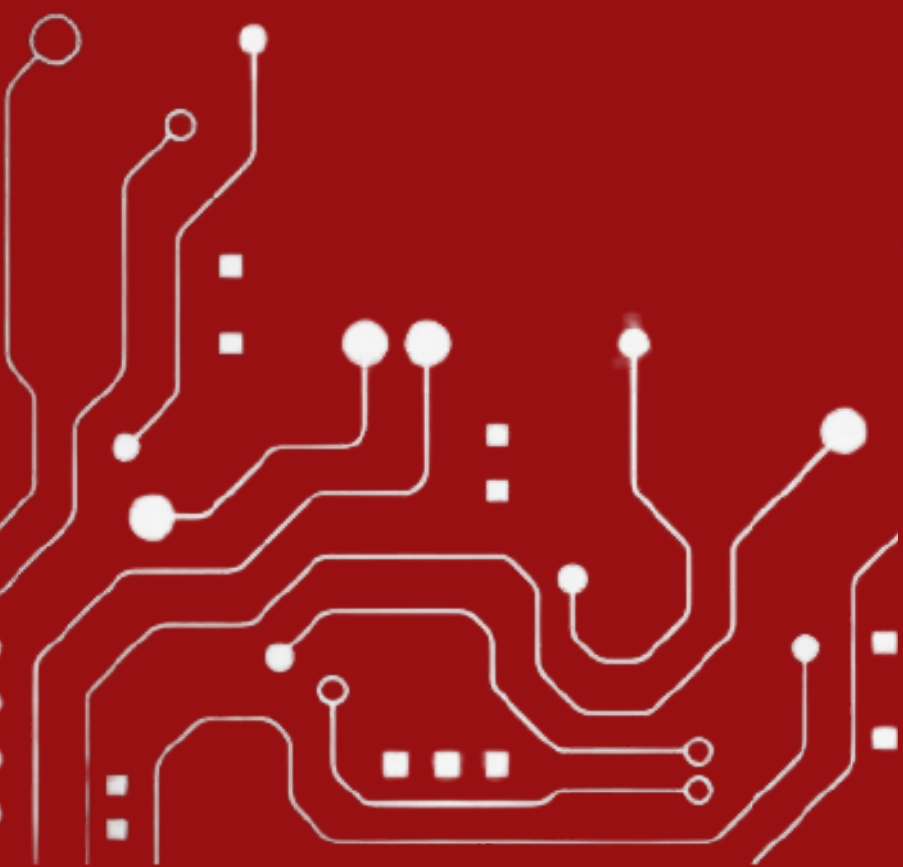


Pandas

Data Analysis with
Python



Agenda

01 Introduction What is Pandas, why it matters, the ecosystem

03 Exploring Data head, info, describe, shape, dtypes

05 Filtering Boolean, isin, between, query, where, mask

07 Data Cleaning Missing values, duplicates, types, strings, outliers

02 Core Structures Series, DataFrames, creating & loading data

04 Accessing Data Columns, loc, iloc, at, iat & sorting

06 Transforming apply, map, transform, assign

08 Visualization Charts with .plot() — bar, hist, scatter, pie, saving

Introduction

What is Pandas & why it matters

What is Pandas?

Open-Source Python Library

First released in 2008 by Wes McKinney. Powers data manipulation, cleaning, and analysis across all industries.

Built on NumPy

Uses NumPy arrays internally. Operations run in compiled C — orders of magnitude faster than pure Python loops.

Name: Panel Data

From the econometrics term for structured multi-dimensional datasets. Not named after the animal!

Two Core Structures

Series (1D labeled array) and DataFrame (2D table). You'll spend 90% of your time with DataFrames.

```
# Install
pip install pandas

# Import convention
import pandas as pd
```

Works alongside:

NumPy
Fast arrays

Matplotlib
Plotting

Scikit-learn
ML

SQLAlchemy
Databases

Key Use Cases:

Data Wrangling

EDA & Profiling

Feature Engineering

Time-Series Analysis

Reporting & Export

Core Structures

Series, DataFrames & Loading Data

Series vs DataFrame

Series (1D)

A labeled 1-D array — like a single spreadsheet column with an index.

Index	Value
a	10
b	20
c	30
d	40

```
s = pd.Series([10,20,30,40], index=["a","b","c","d"])
s["b"] # 20      s.mean() # 25.0      s[s > 15] # filter
```

- A DataFrame column is a Series
- Both support vectorized ops — no loops needed
- Custom index labels replace 0,1,2...

DataFrame (2D)

A 2-D table with labeled rows and columns — like a spreadsheet or SQL table.

#	Name	Age	Score
0	Ali	25	88
1	Sara	30	95
2	Omar	22	76

```
df = pd.DataFrame({
    "Name":  ["Ali", "Sara", "Omar"],
    "Age":   [25, 30, 22],
    "Score": [88, 95, 76]
})
```

Creating DataFrames — 3 Ways

From a Dictionary

```
df = pd.DataFrame({  
    "Name": ["Ali", "Sara"],  
    "Age":  [25, 30]  
})
```

Keys become column names.
Most readable approach.

From a List of Lists

```
df = pd.DataFrame(  
    [[1, 2], [3, 4]],  
    columns=["A", "B"]  
)
```

Data passed row by row.
Column names added separately.

With Custom Index

```
df = pd.DataFrame(  
    [[1,2],[3,4]],  
    columns=["A","B"],  
    index=["r1","r2"]  
)
```

Replace 0,1,2 index with
meaningful row labels.

Quick inspection: `df.columns` • `df.shape` • `df.dtypes` • `df.index` • `df.info()`

Loading Data

CSV / TSV

```
pd.read_csv("file.csv")  
pd.read_csv("file.tsv", sep="\t")
```

Excel

```
pd.read_excel("file.xlsx")  
pd.read_excel("file.xlsx", sheet_name="Sheet2")
```

JSON

```
pd.read_json("file.json")
```

SQL

```
pd.read_sql(query, connection)
```

Parquet

```
pd.read_parquet("file.parquet") # fast columnar format
```

Useful Parameters for read_csv():

sep=';' → Change delimiter

usecols=['A','B'] → Load only specific columns

nrows=100 → Load first N rows only

index_col='ID' → Use a column as the index

parse_dates=['Date'] → Auto-parse date columns

na_values=['N/A','-'] → Extra NaN markers

Saving: `df.to_csv("out.csv", index=False)`

Exploring Your Data

head, info, describe & shape

Exploring Data — Essential Commands

`df.head(n)`

First n rows (default 5). Always run this right after loading your data.

`df.tail(n)`

Last n rows. Check if data ends correctly, especially after filtering.

`df.sample(n)`

n random rows. Use `random_state=42` for reproducibility.

`df.info()`

Column names, dtypes, non-null counts, and memory usage — key diagnostic tool.

`df.describe()`

Count, mean, std, min/max, and quartiles for all numeric columns.

`df.shape`

Returns (rows, cols) tuple — quickest size check before any operation.

`df.nunique()` unique count/col | `df['col'].unique()` all unique values

Accessing Data

Columns, rows, loc & iloc indexing

Accessing Columns & Rows

Column Access

Single column → Series

```
df["Price"]
```

Multiple columns → DataFrame

```
df[["Price", "Units"]]
```

Dot notation (no spaces)

```
df.Price
```

Getting Raw Values

```
df["Price"].values    # → NumPy array (fastest, zero-copy)  
df["Price"].tolist()  # → Python list
```

loc vs iloc

loc — Label-Based (INCLUSIVE slicing)

```
df.loc[0]                # row with index label 0
```

```
df.loc[0, "Price"]        # specific cell value
```

```
df.loc[0:3, "Price":"Units"] # slice (inclusive both ends)
```

```
df.loc[df["Price"] > 5]    # boolean filter
```

iloc — Position-Based (EXCLUSIVE end, 0-indexed)

```
df.iloc[0]               # first row
```

```
df.iloc[-1]              # last row
```

```
df.iloc[0:5, 0:3]        # rows 0-4, cols 0-2 (end EXCLUSIVE)
```

Sorting & Indexing

Before `sort_values("Price")`

Type	Price	Units
Latte	4.50	30
Espresso	2.80	55
Mocha	5.20	20
Tea	1.50	40



After `sort_values("Price")`

Type	Price	Units
Tea	1.50	40
Espresso	2.80	55
Latte	4.50	30
Mocha	5.20	20

Sort Commands:

Ascending (default)

```
df.sort_values("Price")
```

Descending

```
df.sort_values("Price", ascending=False)
```

Multiple columns

```
df.sort_values(["Units","Price"], ascending=[False,True])
```

By index

```
df.sort_index()
```

Reset index after sort

```
df.reset_index(drop=True, inplace=True)
```

Filtering Data

Boolean, isin, query, where & mask

Filtering — Boolean Conditions

Boolean indexing returns rows where a condition is True:

Type	Price	Units	
Latte	4.50	42	✓
Espresso	2.80	55	✓
Mocha	5.20	18	✗
Tea	1.50	40	✓
Matcha	4.80	35	✓

```
df[df["Units"] > 30]
```

Single Condition

```
df[df["Price"] > 4]
```

AND — both must be true

```
df[(df["Price"] > 4) & (df["Units"] > 30)]
```

OR — either must be true

```
df[(df["Price"] > 4) | (df["Units"] > 50)]
```

NOT — negate a condition

```
df[~(df["Type"] == "Tea")]
```

Filter then select column

```
df[df["Price"] > 4]["Type"]
```

Advanced Filtering — isin, between, str, query

isin() — match a list

```
df[df["Type"].isin(["Latte","Espresso"])]  
df[~df["Type"].isin(["Tea"])] # exclude Tea
```

between() — numeric range (inclusive)

```
df[df["Price"].between(3.0, 6.0)]  
# = (df["Price"] >= 3.0) & (df["Price"] <= 6.0)
```

str.contains() — text match

```
df[df["Name"].str.contains("Ali")]  
df[df["Email"].str.endswith("@gmail.com")]
```

query() — SQL-like syntax

```
df.query("Price > 4 and Type == 'Latte'")  
threshold=4.0; df.query("Price > @threshold")
```

isna() / notna() — missing value filters

```
df[df["Notes"].isna()] # rows with NaN  
df[df["Notes"].notna()] # rows without NaN
```

Conditional Replacement — where() & mask()

Two functions to conditionally keep or replace values:

where() — Keep where TRUE

Keeps values where condition is TRUE. Replaces with NaN (or other=) where FALSE.

```
# Keep Price where > 4, else NaN
df["Price"].where(df["Price"] > 4)

# Keep where > 4, replace with 0
df["Price"].where(df["Price"] > 4, other=0)
```

mask() — Replace where TRUE (opposite)

Replaces values where condition is TRUE. Keeps values where FALSE. Exact inverse of where().

```
# Cap outliers: replace Price > 10 with 10
df["Price"].mask(df["Price"] > 10, other=10)

# Replace all negatives with 0
df.mask(df < 0, other=0)
```

	where()	mask()
Condition TRUE	KEEP the value	REPLACE value
Condition FALSE	REPLACE with NaN / other=	KEEP the value

Pro tip: mask() is the exact inverse of where().

Transforming Data

apply, map, transform & assign

Adding, Removing & Transforming Columns

Adding Columns

Constant value

```
df["Tax"] = 0.15
```

Computed from existing

```
df["Revenue"] = df["Units"] * df["Price"]
```

apply() — element-wise

```
df["Cat"] = df["Price"].apply(  
    lambda x: "High" if x > 5 else "Low")
```

assign() — chainable

```
df = df.assign(  
    Tax=df.Price * 0.15,  
    Total=df.Price * 1.15)
```

Removing: `df.drop("Tax", axis=1)` • Always `df2 = df.copy()` before mutating!

map() & transform()

map() — element-wise replacement / function

```
# Replace values using a dictionary  
df["Grade"] = df["Score"].map(  
    {90:"A", 80:"B", 70:"C"})  
  
# Apply a function to every element  
df["Type"].map(str.upper)
```

transform() — group-aware, preserves shape

Adds a group statistic back as a new column without collapsing rows:

```
# Add mean price per type as a new column  
df["TypeMean"] = df.groupby("Type")["Price"]  
    .transform("mean")  
  
# Z-score per group  
df["ZScore"] = df.groupby("Type")["Price"]  
    .transform(lambda x: (x - x.mean()) / x.std())
```

Data Cleaning

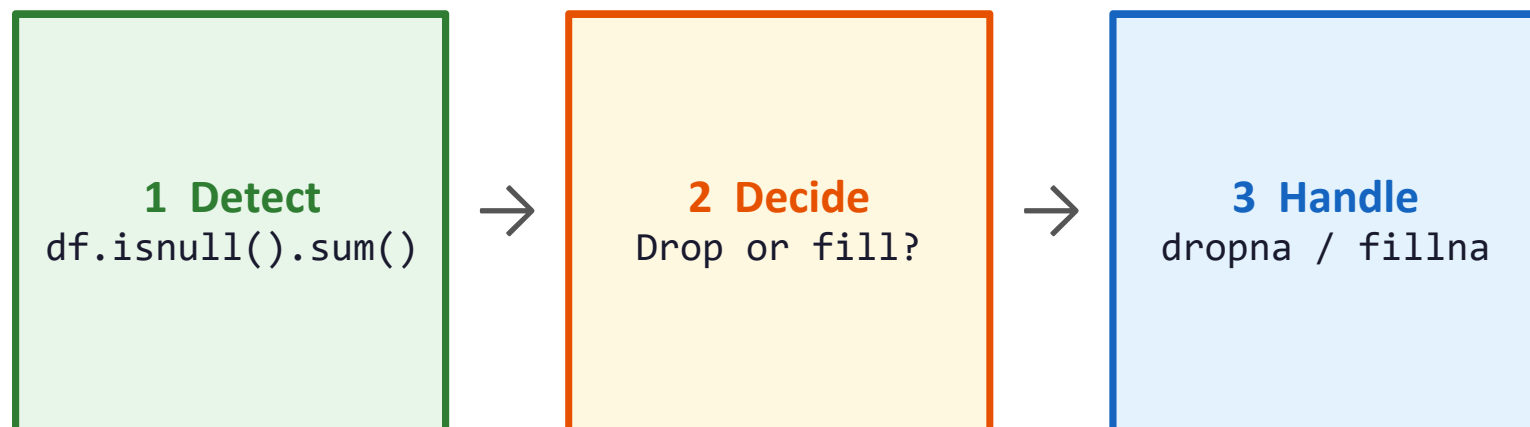
Missing values, duplicates, types & strings

Data Cleaning — Missing Values (NaN)

A messy dataset:

Name	Age	Score
Ali	25	88.0
Sara	NaN	95.0
Omar	22	NaN
Nour	30	72.0

3-Step Process:



Drop rows / columns

```
df.dropna()           # any NaN
df.dropna(subset=["Age"]) # only if Age is NaN
df.dropna(axis=1, thresh=3) # drop cols < 3 non-NaN
```

Fill with a value

```
df.fillna(0)           # all NaN → 0
df["Age"].fillna(df["Age"].mean(), inplace=True)
df.fillna(method="ffill") # forward fill
```

Interpolate

```
df["Score"].interpolate() # linear
df["Score"].interpolate(method="polynomial", order=2)
```

Data Cleaning — Duplicates, Types & Strings

Duplicates

```
df.duplicated().sum()
df.drop_duplicates(inplace=True)
df.drop_duplicates(subset=["Name"], keep="first")
```

.str Accessor — String Operations

Use `df['col'].str.method()` for text columns:

Cleaning

```
df["Name"].str.strip()
df["Name"].str.lower() / .upper() /
.title()
```

Searching

```
df["Email"].str.contains("@gmail")
df["ID"].str.startswith("EMP")
```

Type Conversion

```
df["Age"] = df["Age"].astype(int)
df["Date"] = pd.to_datetime(df["Date"])
df["Price"] = pd.to_numeric(df["Price"], errors="coerce")
df.rename(columns={"old":"new"}, inplace=True)
```

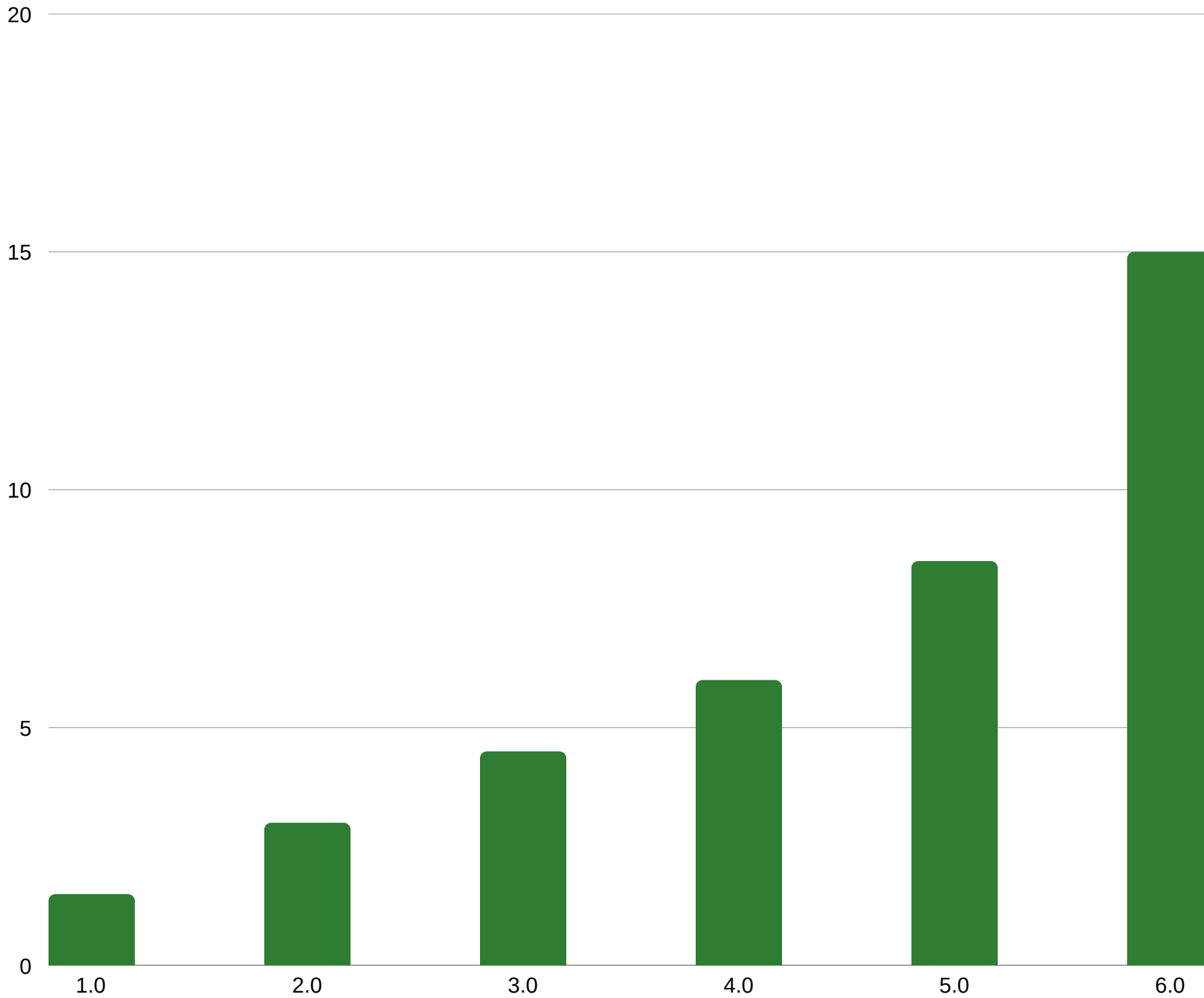
Extraction

```
df["Email"].str.split("@").str[1]  #
domain
df["Code"].str.extract(r"(\d+)")    #
regex
```

Replacement

```
df["Phone"].str.replace("-",
",", regex=False)
df["Name"].str.replace(" ", "_")
```

Data Cleaning — Outlier Detection



IQR Method — Robust Detection

```
Q1 = df["Price"].quantile(0.25)
Q3 = df["Price"].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df["Price"] < Q1-1.5*IQR) |
               (df["Price"] > Q3+1.5*IQR)]
```

Clip to a valid range

```
# Values below 0.5 → 0.5, above 20 → 20
df["Price"] = df["Price"].clip(lower=0.5, upper=20.0)
```

Validate with assertions

```
assert df["Age"].between(0, 120).all()
assert df.isnull().sum().sum() == 0
assert (df["Price"] > 0).all()
```

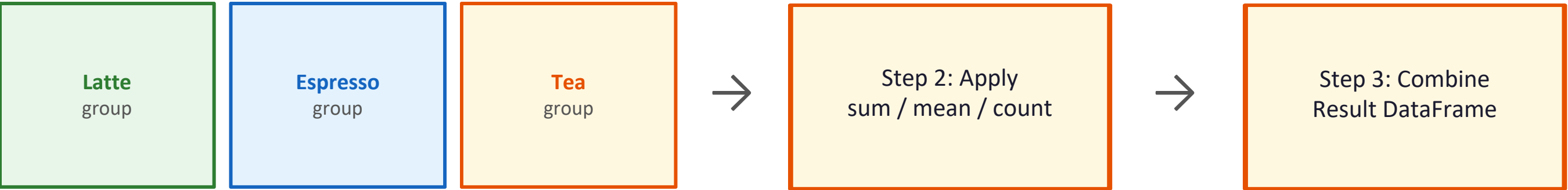
```
df.corr() → correlation matrix • df["col"].value_counts(normalize=True)
```

GroupBy & Merging

Aggregate, pivot & join DataFrames

GroupBy — Split → Apply → Combine

Step 1: Split



Type	Avg Price	Total Units
Espresso	2.80	55
Latte	4.50	42
Tea	1.50	40

```
df.groupby("Type")["Price"].mean()           # mean price per type

df.groupby("Type").agg(
    Total    =("Units", "sum"),
    AvgPrice=("Price", "mean"),
    Count    =("Units", "count")
)                                             # named multi-aggregation

pd.pivot_table(df, values="Revenue", index="Day",
               columns="Type", aggfunc="sum", fill_value=0)
```

Merging & Joining DataFrames

INNER JOIN

Only rows matching in BOTH tables

```
pd.merge(orders, customers,  
         on="CustomerID", how="inner")
```

LEFT JOIN

All from left, NaN where right has no match

```
pd.merge(orders, customers,  
         on="CustomerID", how="left")
```

RIGHT JOIN

All from right, NaN where left has no match

```
pd.merge(orders, customers,  
         on="CustomerID", how="right")
```

OUTER JOIN

All rows from BOTH, NaN for non-matches

```
pd.merge(orders, customers,  
         on="CustomerID", how="outer")
```

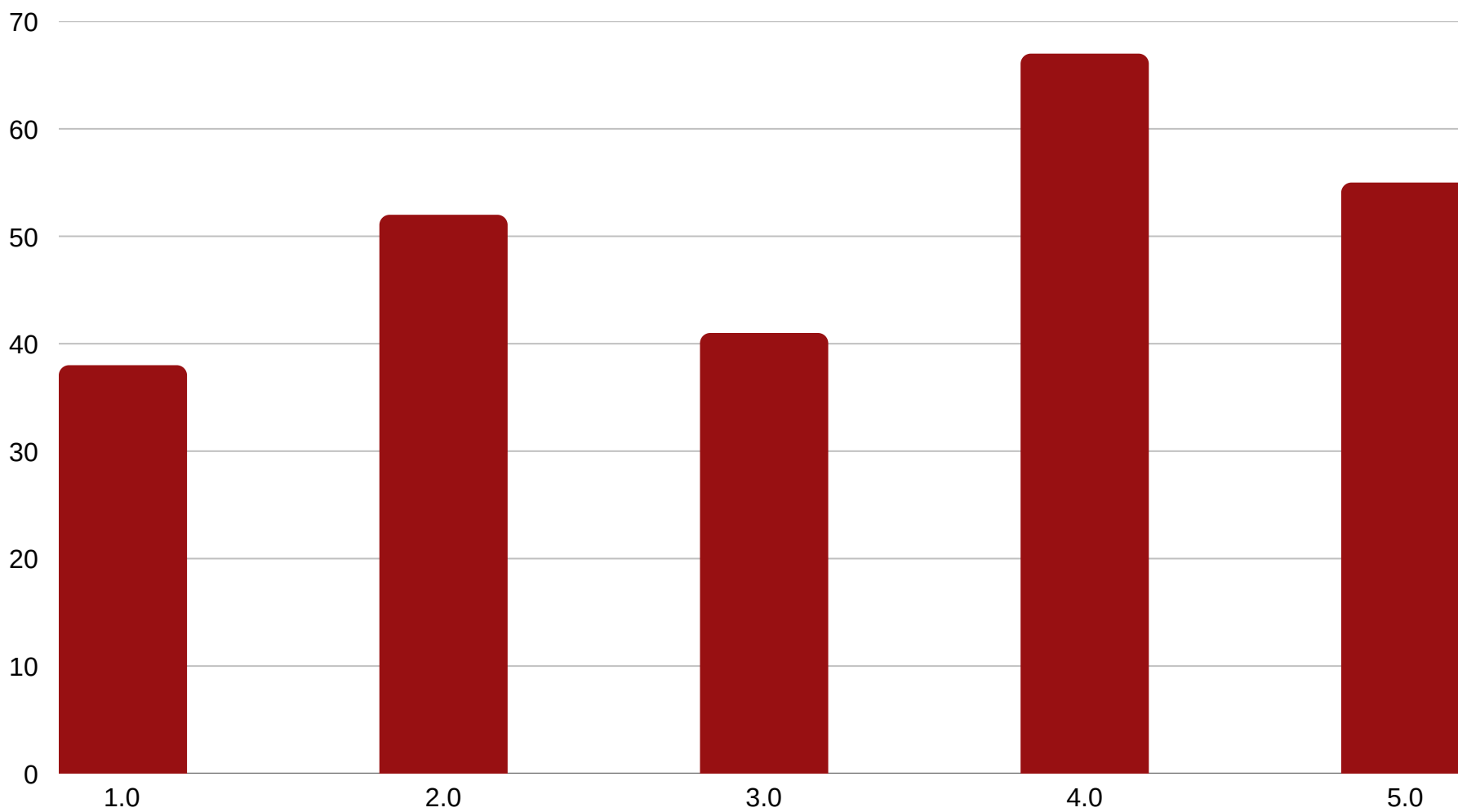
concat() Stack DataFrames vertically or horizontally: `pd.concat([df1,df2,df3], ignore_index=True)`

Visualization

Charts with Pandas .plot()

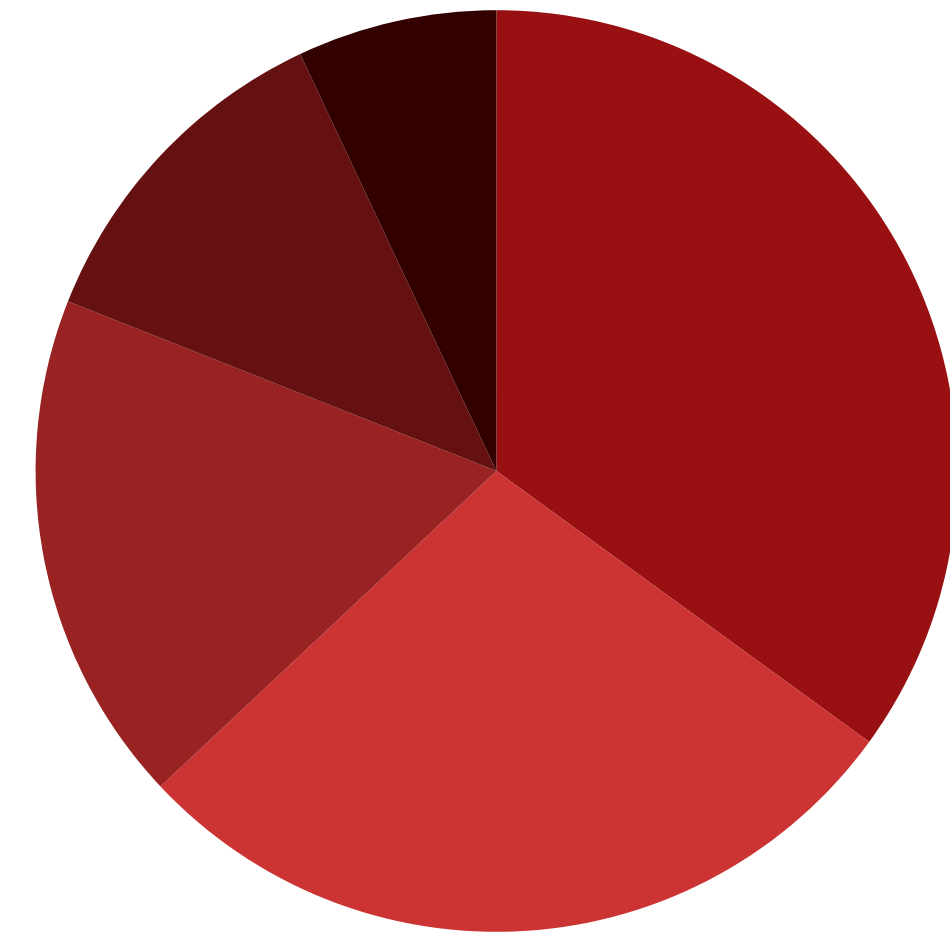
Visualization — .plot() in One Line

Pandas wraps Matplotlib. Always import matplotlib.pyplot as plt



```
df.groupby("Day")["Units"].sum()
    .plot(kind="bar", color="#981012",
          title="Units Sold per Day")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

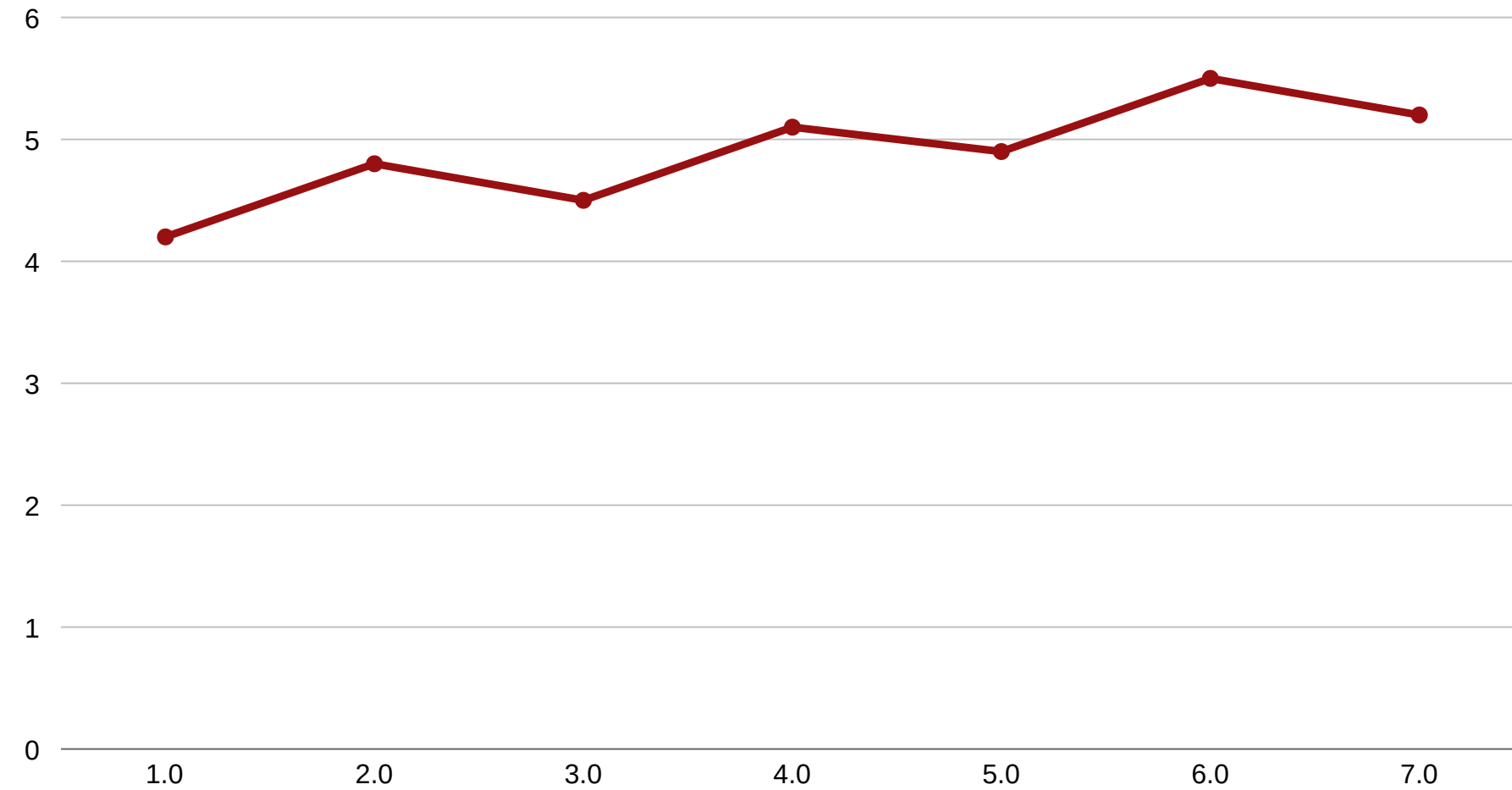
● Latte ● Espresso ● Mocha ● Tea
● Other



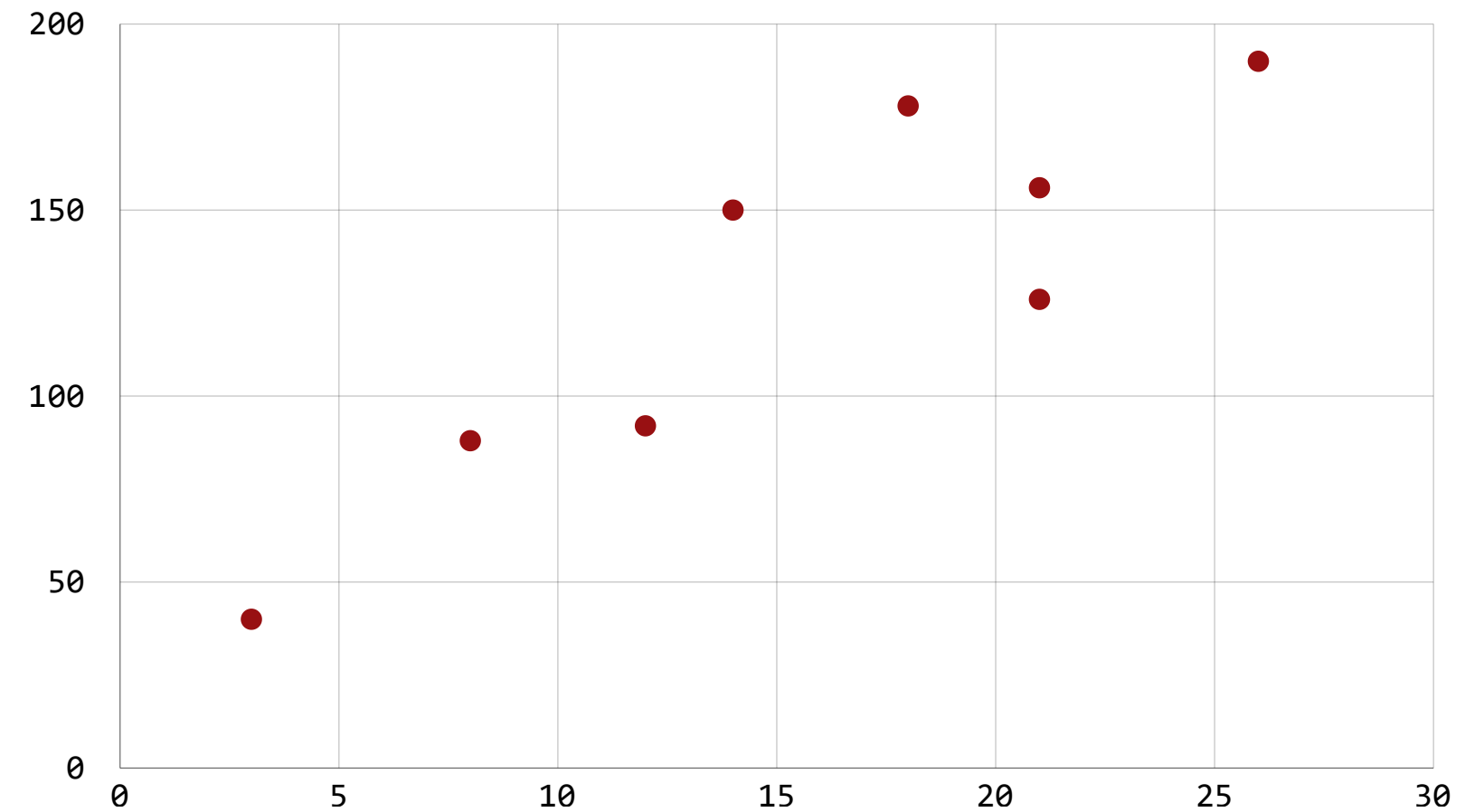
```
df.groupby("Type")["Revenue"].sum()
    .plot(kind="pie",
          autopct="%1.1f%%",
          title="Revenue Share")
plt.ylabel("")
plt.show()
```

kind= "bar" · "hist" · "scatter" · "box" · "line" · "area" · "pie"

Visualization — Line, Scatter & Saving Charts



```
df.groupby("Day")["Price"].mean()
    .plot(kind="line", linewidth=2,
          color="#981012", marker="o",
          title="Avg Price Over Week")
plt.show()
```



```
df.plot(kind="scatter",
        x="Price", y="Units",
        color="#981012", alpha=0.7,
        title="Price vs Units Sold")
plt.show()
```

Customization & Saving

```
plt.title("My Chart") • plt.xlabel("X Label") • plt.ylabel("Y Label")
plt.figure(figsize=(10,5)) • plt.savefig("chart.png", dpi=150, bbox_inches="tight")
fig, axes = plt.subplots(1, 2, figsize=(12,5)) # side-by-side subplots
```

Thank You

